

[Edit](#)

Control Git Behavior with Config and Ignore Files

Challenge Overview

Understand the scenario

You are a DevOps Engineer for Hexelo, an organization that needs to manage files and Git settings.

In this Challenge Lab, you will configure your Git environment and then create and test an alias. First, you will create a local .gitignore file to manage which files are tracked, add rules to the file, and test to ensure it is working. You will also test the "already tracked" scenario. Next, you will explore global vs. local git configuration settings and modify local settings. Finally, you will create a global ignore file with OS metadata ignore rules.

Navigating the Challenge Lab

Unable to process content from <https://raw.githubusercontent.com/LODSCContent/Challenge-V3-Framework/main/Templates/Environments/None.md>

🔗 ► Quick tips for navigating the Challenge Lab instructions.

[New to Challenge Labs? Click here to learn more.](#)

Configure the Git environment and use aliases

Hints Enabled

No Yes

- Sign in to the virtual machine as **LabUser** by using **T** **Passw0rd!** as the password.
- Enter your GitHub Username, Email Address, Password, and GitHub Account URL into the following associated textboxes:

GitHub Username

GitHub Email Address

GitHub Password

GitHub URL

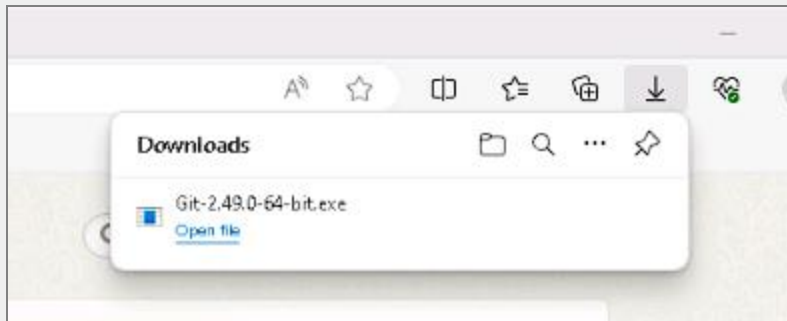
- If necessary, go to **T** <https://git-scm.com/download/win>, and then download and install **Git** by using the default options.

💡 Expand this hint for guidance on installing Git.

- On the Windows taskbar, in Search, enter **Edge**, and then select **Microsoft Edge** to open a new browser.
- In the Address bar, enter <https://git-scm.com/download/win>, and then press **Enter**.
- On the **git** Download for Windows page, in Downloads, select **Click here to download**.



- Once the download completes, select **Open file**.



- On the *User Account Control - Do you want to allow this app to make changes to your device?* dialog, select **Yes**.
- On the Git Setup page, leave *Only show new options* selected, and then select **Install**.



Git 2.49.0 Setup

Information
Please read the following important information before continuing.

When you are ready to continue with Setup, click Next.

GNU General Public License
Version 2, June 1991

Copyright (C) 1989, 1991 Free Software Foundation, Inc.
59 Temple Place - Suite 330, Boston, MA 02111-1307, USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change

<https://gitforwindows.org/>

☒ Only show new options

Install **Cancel**

- Once the Wizard is complete, select **Launch Git Bash**, deselect **View Release Notes**, and then select **Finish**.

 Always verify Git is installed before running Git Bash commands by typing `git --version` in a terminal.

 Want to learn more? Review the documentation on [installing git](#).

- Set your Git Identity by using the `git config` command and using `<Name>` as the **username** and `<GithubEmail>` as the **email address**.

 Expand this hint for guidance on setting your Git Identity. 

- In the Git Bash Terminal, run the following commands to set your Git identity-press **Enter** on your keyboard after entering a command.

```
git config --global user.name "<Name>"
```

```
git config --global user.email "<GithubEmail>"
```

 Want to learn more? Review the documentation on [setting your Git Identity](#).

- Create a Git project folder named `Lab6` in the `D:\LabFiles` directory and initialize a new Git repository in the `Lab6` folder.

💡 Expand this hint for guidance on creating a project folder and initializing the repository. ^

- In the Git Bash terminal, make sure you are in the **LabFiles** directory by running `cd /d/LabFiles`.
- Create a new directory named `Lab6` by running `mkdir Lab6`.
- Change to the **Lab6** directory by running `cd Lab6`.
- Initialize a new Git repository by running `git init`.

- Make your first commit by creating a file named `README.md` by using the `echo` command, and then stage and commit the new file by using the `git add` and `git commit` commands.

💡 Expand this hint for guidance on making your first commit. ^

- Create a `README.md` file by running `echo "# Lab6" > README.md`.
- Stage the file using `git add README.md`.
- Commit the file using `git commit -m "Initial commit"`.

```
LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$ git commit -m "Initial commit"
[master (root-commit) 81778a3] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 README.md

LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$
```

- In the Git Bash terminal, run the following command to display your current global configuration values:

```
git config --global --list
```

📄 These settings apply to all repositories on your system unless overridden by repo-level configuration.

- Using the alias `git lg` and the `git config` command, create a short alias to simplify commonly used Git commands by setting a global alias for Git log.

💡 Expand this hint for guidance on setting a global alias for Git log. ^

- In Git Bash, run the following command to set a global alias for Git Log:

```
git config --global alias.lg "log --oneline --graph --all"
```

- In Git Bash, run `T` `git lg` to use the alias, and then press **Enter**.

```
LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$ git lg
* 81778a3 (HEAD -> master) Initial commit

LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$ |
```

💡 Want to learn more? Review the documentation on [setting a global alias](#).

📄 Aliases reduce repetitive typing and help enforce consistency across teams.

Check your work

Verify

Understand and use .gitignore

Hints Enabled

No Yes

- In the `D:\LabFiles\Lab6` folder, prevent Git from tracking unwanted files by creating a `.gitignore` file.

💡 Expand this hint for guidance on creating the `.gitignore` file.

- In Git Bash, if necessary, change to the **Lab6** folder by running `cd /d/LabFiles/Lab6`.
- Create the file by running `touch .gitignore`.
- Confirm it exists by running `ls -la` and verifying **.gitignore** appears.

The screenshot shows two parts: a terminal window and a Windows File Explorer window.

Terminal Window:

```
LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$ ls -la
total 5
drwxr-xr-x 1 LabUser 197121 0 Dec 16 20:20 ./
drwxr-xr-x 1 LabUser 197121 0 Dec 16 20:18 ../
drwxr-xr-x 1 LabUser 197121 0 Dec 16 20:19 .git/
-rw-r--r-- 1 LabUser 197121 0 Dec 16 20:20 .gitignore
-rw-r--r-- 1 LabUser 197121 7 Dec 16 20:19 README.md

LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$
```

File Explorer Window:

The File Explorer shows the contents of the `Lab6` folder. The address bar indicates the path: `This PC > Local Disk (D:) > LabFiles > Lab6`.

Name	Date modified
.gitignore	12/16/2025 8:20 PM
README	12/16/2025 8:19 PM

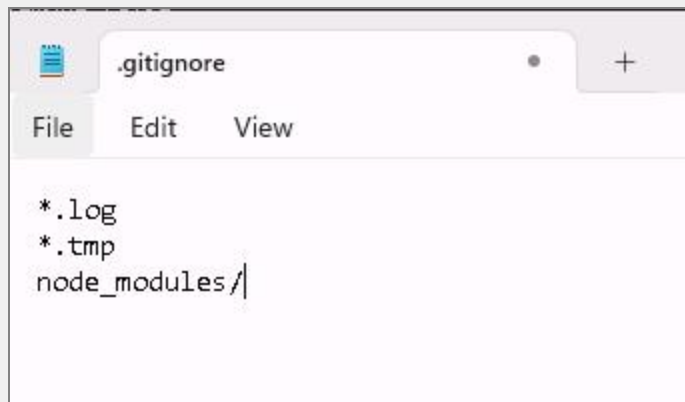
💡 Want to learn more? Review the documentation on [gitignore](#).

- Add ignore rules for log files, temporary files, and the `node_modules` folder, and then save the `.gitignore` file.

💡 Expand this hint for guidance on adding ignore rules. ^

- In File Explorer, open `D:\LabFiles\Lab6`.
- Right-click the `.gitignore` file, select **Open with**, and then select **Notepad**.
- Replace the contents of the file with the following lines (make sure the first two lines start with an asterisk):

📄
*.log
*.tmp
node_modules/



- Save and close the file.

📄 Prefer terminal-only? Overwrite the file with the correct rules by running:

```
printf "%s\n" "*.log" "*.tmp" "node_modules/" > .gitignore
```

📄 The rules from the `.gitignore` file will apply locally only.

- In Git Bash, run `git check-ignore -v error.log` and confirm the output references ***.log** (not `.log`).

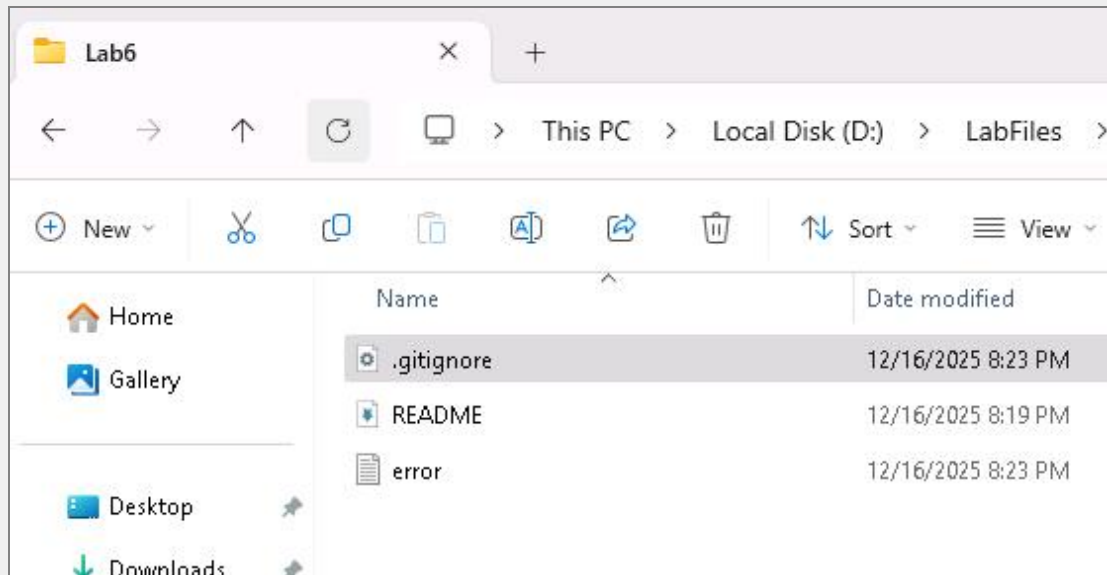
💡 Want to learn more? Review the documentation on the [check-ignore](#) command.

📄 The output should display `*.log` as well as `error.log`.

- Verify that Git is using the ignore rules by creating and testing a log file named `error.log`, and then checking the ignore match.

💡 Expand this hint for guidance on testing ignore rules.

- Create a test log file by running `echo "Temporary log" > error.log`.



- Confirm Git ignores it by running `git check-ignore -v error.log`.
- You should see output that references the `.gitignore` file and `*.log`.
- Run `git status` and confirm **error.log** does **NOT** appear as an untracked file.

```
LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

nothing added to commit but untracked files present (use "git add" to track)
LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$ |
```

⚠ If `git check-ignore -v error.log` prints no output, your ignore rule is **incorrect**.

Confirm the rule is `*.log` (not `.log`) by running `cat .gitignore`.

- Stage and commit the `.gitignore` file with the message `"Add .gitignore file"` so the exclusion rules are shared with collaborators.

 Expand this hint for guidance on committing the .gitignore file.

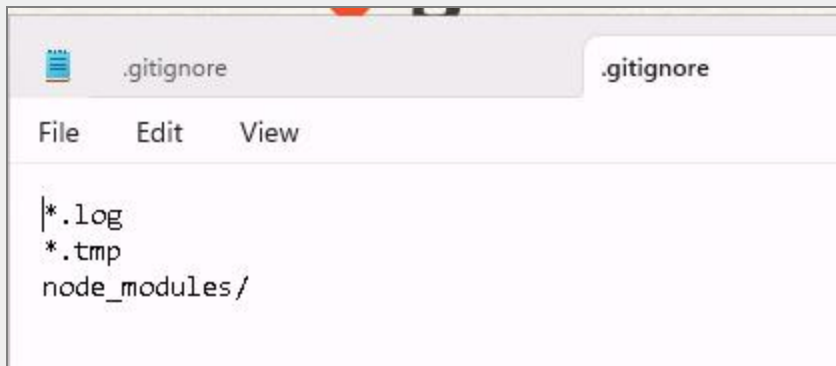
- Stage the ignore file by running `git add .gitignore`.
- Commit the file by running `git commit -m "Add .gitignore file"`.
- Run `git status` and confirm the repository is clean (no pending changes).

 Files already tracked by Git will not be ignored unless removed from the index first.

- Demonstrate the "already tracked" scenario by intentionally tracking a `.tmp` file named `test.tmp` by using the `git add` command with the `-f` (`-force`) parameter.
 - Commit the file with the message: `"Add test.tmp"`.

 Expand this hint for guidance on demonstrating the already tracked scenario.

- Create a temporary file named `test.tmp` by running `echo "should not track" > test.tmp`.
- Stage the `test.tmp` file by running `git add -f test.tmp`.
- Verify:
 - Run `git status` (`test.tmp` should now be tracked).
- Commit the change by running `git commit -m "Add test.tmp"`.
- Open `.gitignore` to confirm it still includes `*.tmp` or run the command `cat .gitignore`.



```
.gitignore
File Edit View
*.log
*.tmp
node_modules/
```

 The `-force` parameter is needed because the file was created after the ignore rule was added to the `.gitignore` file.

If you run the `git add .gitignore` command without the `-force` parameter, you will see an error message.

💡 Want to learn more? Review the documentation on using [git add with the -force parameter](#).

- Stop tracking the `test.tmp` file by using the `git rm` command and keep it locally on your system by using the `--cached` parameter (without the `--cached` parameter, Git would delete the file entirely).
 - Commit the change with the message: `"Stop tracking test.tmp"`.

💡 Expand this hint for guidance on how to stop tracking a file. ^

- Stop tracking the file (but keep it locally) by running `git rm --cached test.tmp`.
- Commit the change by running `git commit -m "Stop tracking test.tmp"`.
- Verify:
 - Run `git status` (the working tree should be clean).
 - Run `git status --ignored` and confirm **test.tmp** appears under **Ignored files**.

💡 Want to learn more? Review the documentation on [how to stop tracking a file](#) in git.

Check your work

Verify

Explore advanced configuration practices

Hints Enabled

No Yes

- Review Git's configuration precedence for the system settings (lowest priority) by running the following command:

```
git config --system --list
```

 Expand this hint for guidance on exploring Git's configuration precedence. 

- In Git Bash, run the `git config --system --list` command to explore Git's config hierarchy (local > global > system), and then press **Enter**.

```
LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$ git config --system --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master

LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$ |
```

 Want to learn more? Review the documentation on the [git config](#) command.

- Compare the global list to the local list by using the different parameters `--global` and `--local` with the command `git config`.

💡 Expand this hint for guidance on comparing lists.

- Review the User-global settings by running the following command:

```
git config --global --list
```

- Review the Repository-specific settings (highest priority) by running the following command:

```
git config --local --list
```

💡 Want to learn more? Review the documentation on [comparing local and global lists](#) on the git config command page.

- Modify a setting locally and watch it override globally by using the `T` `user.name "Override Name"` parameter with the `T` `git config` command.

💡 Expand this hint for guidance on modifying a setting locally.

- In Git Bash, run `T` `git config user.name "Override Name"`, and then press **Enter**.

```
LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$ git config user.name "Override Name"

LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$ git config --get user.name
Override Name

LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$ |
```

📄 The order of precedence is local > global > system.

Verify the change by running `T` `git config --get user.name` to retrieve the local `user.name` value.

- Use the following to organize your `T` `.gitignore` for readability by adding comments and blank lines in the file:

```
📄 # Ignore Logs
*.log

# Ignore temp files
*.tmp
```

```
# Ignore dependency folders
node_modules/
```

💡 Expand this hint for guidance on adding comments and blank lines in the `.gitignore` file. ^

- In **Notepad**, edit the `.gitignore` file, update the current text by using the above example, and then select **File | Save**.

💡 Want to learn more? Review the documentation on [adding comments and blank lines in .gitignore](#).

- Confirm the behavior remains unchanged by using the `git status` command.

```
LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

no changes added to commit (use "git add" and/or "git commit -a")
LabUser@Client01 MINGW64 /d/LabFiles/Lab6 (master)
$
```

- Stage and commit the `.gitignore` file by running the following commands:

```
git add .gitignore
```

```
git commit -m "Added comments to .gitignore"
```

📄 Comments help teams understand the purpose behind ignore rules.

- Create a global ignore file for your user account for all repositories by using the `touch` command and the file name of `.gitignore_global`.

💡 Expand this hint for guidance on creating a global ignore file. ^

- In the Git Bash terminal, run the following command to create a global `.gitignore` file:

```
touch ~/.gitignore_global
```

- Configure Git to use the `.gitignore_global` file by using the `git config` command with the `--global` parameter.



Expand this hint for guidance on configuring git to use a global ignore file.



- In Git Bash, run the following command to configure git to use a global ignore file:

```
git config --global core.excludesfile ~/.gitignore_global
```

- Add OS metadata ignore rules to the `.gitignore_global` file by running the following command:

```
printf "%s\n" ".DS_Store" "Thumbs.db" >> ~/.gitignore_global
```

- Confirm the change by running the following command:

```
git config --global core.excludesfile
```

Check your work

Verify

Summary

Congratulations, you have completed the **Control Git Behavior with Config and Ignore Files** Challenge Lab.

You have accomplished the following:

- Configured the Git environment and used aliases.
- Understood and used .gitignore.
- Explored advanced configuration practices.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

 Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to [Challenge Labs feedback](#).

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Track Local Changes and Manage Commits [Guided]
- Create Branches and Merge Changes Safely [Guided]
- Collaborate Remotely and Use Pull Requests [Guided]
- Build a Real Project by Using Git Workflows [Advanced]
- Write Code with GitHub Copilot Suggestions [Guided]
- Control Git Behavior with Config and Ignore Files [Guided]

- [Rebase, Stash, and Apply Commits Precisely \[Guided\]](#)
- [Manage Tasks by Using GitHub Issues and Boards \[Guided\]](#)
- [Automate Testing and CI with GitHub Actions \[Guided\]](#)
- [Review and Refactor Code with Copilot \[Guided\]](#)
- [Implement Release Management and Versioning in GitHub \[Expert\]](#)

